Information Bandwidth of Reinforcement Learning

Lecture Notes on Why RL is Data-Inefficient
and What We Can Do About It

Synthesized from Ord [1], Li [2], Shao et al. [4], and DAPO/GRPO literature

Abstract

These lecture notes provide a treatment of the information-theoretic foundations of reinforcement learning for large language models. We derive the "1 bit per gradient step" bound from first principles, then systematically analyze how this bound changes under realistic training configurations. The analysis draws primarily from Ord's scaling analysis [1] and Li's information-theoretic framework [2], with empirical grounding from DeepSeek-R1 [3] and recent algorithmic developments including GRPO [4], DAPO [5], and GRESO [6].

1. Motivation: The Efficiency Puzzle

Consider the following empirical contrast that motivates everything that follows.

Pre-training a frontier LLM requires approximately 10^{13} tokens (10–15 trillion). Each token provides a supervision signal—the correct next token—and the model learns from every single one.

RL fine-tuning, as exemplified by DeepSeek-R1 [3], uses approximately 4 million rollouts over ∼8,000 gradient steps, with each rollout generating an average of ∼4,000 tokens. That is roughly 16 billion tokens *generated*—but how much *learning signal* do we extract?

The puzzle: DeepSeek-R1's RL phase produces substantial improvements in reasoning ability, yet it processes orders of magnitude fewer "bits of feedback" than pre-training. How do we reconcile this?

Intuition 1.1. The resolution lies in distinguishing between **tokens generated** and **bits of information about the optimal policy**. Pre-training provides ∼3 bits per token (see Section 7). RL with binary rewards provides ∼1 bit per *episode* of thousands of tokens. The information density differs by a factor of 10^3 to 10^6.

Ord [1] articulates this succinctly: "Pre-training via next-token-prediction provides the model with a token worth of information for every token produced during training. In contrast, RL on machine-checkable tasks requires a long chain of thousands or millions of tokens before revealing to the model a single bit of information."

2. The Scalar Advantage Bottleneck

2.1 Setting Up the Problem

Consider a language model π_θ that generates a response (trajectory) $\tau = (a_1, a_2, \ldots, a_T)$ given a prompt. We want to optimize θ to maximize expected reward:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$$

The policy gradient theorem gives:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\nabla_\theta \log \pi_\theta(\tau) \cdot R(\tau)\right]$$

In practice, we use a baseline b to reduce variance, giving the REINFORCE estimator:

$$g = \nabla_\theta \log \pi_\theta(\tau) \cdot \underbrace{(R(\tau) - b)}_{\text{Advantage } A}$$

2.2 The Information-Theoretic Decomposition

Following Li [2], we can formalize the information flow in this gradient. The central question is: how much does gradient g tell us about the optimal policy π^*, given what we already know from training history $\mathcal{H}$?

The gradient g is the product of two terms:

1. $\nabla_\theta \log \pi_\theta(\tau)$: Given the training history $\mathcal{H}$ (which determines θ), this term is **completely determined by** τ. It depends only on the trajectory and current parameters, not on the reward function R.
2. $A = R(\tau) - b$: This is the **only channel** through which information about the reward function R—and hence about the optimal policy π^*—enters the gradient.

Lemma 2.1 (Scalar Advantage Bottleneck)**.** *Let π^* denote the optimal policy for reward function R, and let $\mathcal{H}$ denote the training history. Then:*

$$I(g;\, \pi^* \mid \mathcal{H}) \leq I(A;\, \pi^* \mid \tau, \mathcal{H})$$

where $I(\cdot;\cdot \mid \cdot)$ denotes conditional mutual information.

Proof. The gradient $g = f(\tau, A, \theta)$ is a deterministic function of (τ, A, θ), where θ is determined by $\mathcal{H}$. By the data processing inequality:

$$I(g;\, \pi^* \mid \mathcal{H}) \leq I((\tau, A);\, \pi^* \mid \mathcal{H})$$

The trajectory τ is sampled from π_θ without knowledge of R (given $\mathcal{H}$), so $\tau \perp \pi^* \mid \mathcal{H}$. By the chain rule for mutual information:

$$I((\tau, A);\, \pi^* \mid \mathcal{H}) = I(\tau;\, \pi^* \mid \mathcal{H}) + I(A;\, \pi^* \mid \tau, \mathcal{H}) = 0 + I(A;\, \pi^* \mid \tau, \mathcal{H})$$

□

2.3 The 1-Bit Bound

The advantage A is a scalar random variable. Its mutual information with π^* is bounded by its entropy:

$$I(A;\, \pi^* \mid \tau, \mathcal{H}) \leq H(A \mid \tau, \mathcal{H}) \leq \log_2(B)$$

where B is the number of distinct values A can take.

Key Result

For binary pass/fail rewards ($R \in \{0, 1\}$), we have $B = 2$, giving:

$$I(g; \pi^*) \leq 1 \text{ bit per gradient step}$$

This is an information-theoretic ceiling, not an empirical approximation.

2.4 What This Bound Means

Example 2.2 (Math Problem RL)**.** Consider training a model to solve math problems. We generate one solution, check correctness, and compute a gradient.

The gradient $g \in \mathbb{R}^d$ may have billions of components. But despite this high dimensionality, its *information content about the optimal policy* is at most 1 bit.

The structure of g has the form $g = \nabla_\theta \log \pi_\theta(\tau) \cdot A$ where $A \in \{-b, 1-b\}$. The high-dimensional structure comes from $\nabla_\theta \log \pi_\theta(\tau)$, which is determined by the trajectory the model already generated. The reward contributes exactly one bit: "reinforce this trajectory" or "suppress this trajectory."

Common Misconception

A common confusion: "the gradient has billions of dimensions, so it must carry more than 1 bit." This conflates *dimensionality* with *information content*. A billion-dimensional vector constrained to equal either $+v$ or $-v$ for some fixed v carries exactly 1 bit of information. The dimension count is irrelevant; what matters is the entropy of the distribution over possible gradient values.

Remark 2.3 (Nuance on "1 bit"). The bound $I(g; \pi^*) \leq 1$ bit is a bound on mutual information with the *optimal policy*. This is distinct from:

- The information needed to *represent* the gradient (which could be billions of bits)
- The information the gradient carries about the *trajectory* (which is much higher)
- The practical utility of the gradient for improving the policy

The bound captures specifically how much we learn about *what the optimal policy looks like* from a single reward observation.

3. When the Bound is Tight (and When It's Not)

The 1-bit bound is tight under specific conditions. Understanding when it applies—and when it does not—is essential.

3.1 Conditions for Tightness

The bound is tight when:

1. **Single rollout**: One trajectory per gradient step
2. **Terminal reward**: The entire trajectory receives one scalar score
3. **Binary reward**: Pass/fail with no partial credit
4. **Scalar advantage**: The reward collapses to a single number per trajectory

Li [2] notes: "With terminal rewards only, scalar and per-timestep advantage formulations have the same information ceiling—both $O(1)$ bits. This explains why simple scalar-advantage methods dominate in LLM fine-tuning: the added complexity of per-timestep formulations provides no information-theoretic benefit when rewards are sparse."

3.2 Factors That Increase Information

Each condition above can be relaxed:

Relaxation	Information Gain	Cost
B-level rewards	$\log_2(B)$ bits/advantage	Reward engineering
K rollouts per prompt	$\times K$ multiplier	$K\times$ compute
X prompts per batch	$\times X$ multiplier	Memory
Per-timestep rewards	$\times T$ multiplier	Dense reward design
M reward dimensions	$+M$ additive	Multi-objective setup

4. Non-Binary Rewards: The $\log_2(B)$ Factor

4.1 The Setup

Instead of binary $R \in \{0, 1\}$, suppose rewards take B distinguishable values:

- **5-level rubric**: $R \in \{0, 0.25, 0.5, 0.75, 1\}$, so $B = 5$
- **Code with 10 unit tests**: $R \in \{0, 0.1, 0.2, \ldots, 1\}$, so $B = 11$
- **8-bit quantized**: $R \in \{0, 1, \ldots, 255\}/255$, so $B = 256$

4.2 The Information Bound

The analysis proceeds identically:

$$I(g; \pi^*) \leq \log_2(B) \text{ bits}$$

Reward Type	B	Bits/rollout	vs. Binary
Binary pass/fail	2	1.00	1.0×
Ternary $\{-1, 0, +1\}$	3	1.58	1.6×
5-level rubric	5	2.32	2.3×
10-level rubric	10	3.32	3.3×
Code (10 tests)	11	3.46	3.5×
100-level	100	6.64	6.6×
8-bit quantized	256	8.00	8.0×

Practical Takeaway

Moving from binary to a 5-level rubric more than doubles information bandwidth at zero additional compute cost. Reward shaping is not just about reward quality—it directly increases information throughput.

Ord [1] observes: "There may be productive ways of giving the model more than 1 bit of information per episode. An obvious approach, where applicable, is to score partial success with intermediate reward levels. For example, a switch to a 32-bit floating point reward would give a theoretical ceiling of 32 bits. Though even when there are good ways to score

partial success, it isn't clear that a model could usefully extract more than a few useful bits of information from this."

4.3 The Entropy Constraint

The bound $\log_2(B)$ assumes uniform distribution over the B possible advantage values. In practice:

$$I(A;\, \pi^*) \leq H(A) \leq \log_2(B)$$

with equality only when A is uniformly distributed.

Example 4.1 (Near-Convergence Behavior)**.** Consider a model that succeeds on a problem 99% of the time. The entropy is:

$$H(A) = H(\text{Bernoulli}(0.99)) = -0.99 \log_2(0.99) - 0.01 \log_2(0.01) \approx 0.08 \text{ bits}$$

The model has nearly saturated this problem and extracts almost no information from it.

Key Result

Information is maximized when the model succeeds with probability $p \approx 0.5$ (for binary rewards) or when rewards are uniformly distributed (for multi-level). This provides an information-theoretic justification for curriculum learning: training on problems at the frontier of the model's ability maximizes bits per rollout.

4.4 Continuous Rewards and Practical Limits

In principle, a float16 reward has 2^{16} distinguishable values. This overstates useful information for two reasons:

Gradient noise floor. Stochastic training means the model cannot distinguish reward differences smaller than the effective gradient noise. If gradient noise has standard deviation σ_g and reward-to-gradient sensitivity is $\partial g/\partial R$, the effective resolution is:

$$\Delta R_{\min} \sim \frac{\sigma_g}{|\partial g/\partial R|}$$

Reward noise. If the reward is a noisy estimate of ground truth, Shannon's channel capacity applies. For signal power P and noise power σ^2:

$$C = \frac{1}{2} \log_2 \left(1 + \frac{P}{\sigma^2}\right) \text{ bits}$$

Empirically, effective information from continuous rewards appears to be 3–8 bits, not 16 or 32.

5. Multiple Rollouts: The GRPO Regime

5.1 The Setup

Modern RL for LLMs uses **Group Relative Policy Optimization (GRPO)** [4]:

- Sample X prompts
- For each prompt i, generate K rollouts $\{\tau_{i,1}, \ldots, \tau_{i,K}\}$

- Compute rewards $\{r_{i,1}, \ldots, r_{i,K}\}$
- Compute advantages via z-normalization within each group:

$$A_{i,j} = \frac{r_{i,j} - \mu_i}{\sigma_i + \epsilon}, \quad \mu_i = \frac{1}{K}\sum_j r_{i,j}, \quad \sigma_i^2 = \frac{1}{K}\sum_j (r_{i,j} - \mu_i)^2$$

- Aggregate gradient:

$$g = \sum_{i=1}^{X}\sum_{j=1}^{K} \nabla_\theta \log \pi_\theta(\tau_{i,j}) \cdot A_{i,j}$$

GRPO eliminates the need for a separate critic model by using group-level statistics as baselines. This simplification, introduced in DeepSeekMath [4], has become the dominant approach for RL fine-tuning of LLMs.

5.2 Information Analysis

GRPO uses **per-rollout advantages**: each rollout j gets its own $A_{i,j}$ multiplied by its own gradient direction.

For binary rewards with success probability p_i on prompt i:
Lemma 5.1 (Per-Prompt Information)**.** *The information from prompt i with K rollouts is bounded by:*

$$I(g_i; \pi^* \mid \mathcal{H}) \leq K \cdot H(p_i)$$

where $H(p) = -p\log_2 p - (1-p)\log_2(1-p)$ is binary entropy.

Proof. Each of the K rollouts provides an independent Bernoulli(p_i) outcome. The sufficient statistic is not just the success count (which would give $\log_2(K+1)$ bits) but the *identity of which rollouts succeeded*. Since different rollouts take different trajectories, knowing which specific trajectory succeeded provides credit-assignment information. The total is K independent binary random variables with entropy $H(p_i)$ each. □

Summing over the batch:

Key Result

For a batch of X prompts with K rollouts each, all at optimal difficulty ($p_i = 0.5$):

$$\boxed{I_{\max} = X \cdot K \text{ bits per gradient step}}$$

5.3 DeepSeek-R1 Configuration

According to the DeepSeek-R1 technical report [3], the RL training used:

- 32 prompts per training step
- 16 rollouts per prompt
- Maximum length 32,768 tokens; average length ∼4,000 tokens
- Approximately 8,000 gradient steps total

Configuration	X	K	Bits/step
Minimal GRPO	8	8	64
DeepSeek-R1	32	16	512
Typical GRPO	128	16	2,048
Large-scale	512	32	16,384

Example 5.2 (DeepSeek-R1 Information Budget)**.** DeepSeek-R1 trained for approximately 8,000 gradient steps with 512 rollouts per step. Total information budget:

$$8{,}000 \times 512 = 4.1 \text{ million bits} \approx 500 \text{ KB}$$

For comparison, a rank-64 LoRA adapter on a 7B model with target modules has roughly 10M trainable parameters at 16 bits each $\approx$ 160 million bits of capacity. The RL information budget (4.1M bits) is much smaller, consistent with the observation that RL fine-tuning makes relatively small adjustments to the pre-trained model.

5.4 The Degenerate Group Problem

Not all groups provide information.

Definition 5.3 (Degenerate Group)**.** A group is **degenerate** if all K rollouts receive the same reward (all pass or all fail).

For a degenerate group:

- $\mu_i = r_{i,j}$ for all j, hence $\sigma_i = 0$
- $A_{i,j} = 0$ for all j (with ϵ-stabilization, the advantages are negligible)
- Gradient contribution $g_i \approx 0$
- **Information: exactly 0 bits**

The probability of degeneracy for a prompt with success probability p:

$$P(\text{degenerate}) = p^K + (1-p)^K$$

p	$K = 4$	$K = 16$	$K = 64$
0.5	12.5%	0.003%	≈ 0
0.7	26.5%	3.3%	0.001%
0.9	68.6%	18.5%	0.1%
0.95	81.9%	44.0%	3.7%

Practical Takeaway

At $p = 0.9$ with $K = 16$, nearly 1 in 5 groups contribute zero information—wasted compute. GRESO [6] demonstrated that predicting and skipping low-value prompts before rollout achieves up to 2.4× rollout speedup and 2.0× overall training speedup with no accuracy loss. DAPO [5] introduced Dynamic Sampling to filter zero-variance prompts and refill batches with informative data.

5.5 Diminishing Returns from Larger K

Increasing K has diminishing returns:

1. Marginal information per additional rollout decreases (sampling from the same distribution)
2. Memory constraints mean larger K implies fewer prompts X per batch
3. Wu et al. [7] showed that 2-GRPO retains 98.1% of the performance of 16-GRPO while requiring only 12.5% of the rollouts and 21% of training time

The information-optimal strategy is often: **more prompts at moderate** K rather than fewer prompts at large K.

6. Mixed-Domain Batches and Vector Rewards

6.1 Heterogeneous Reward Functions

Real training batches mix prompts from different domains:

- **Math**: Binary correctness, $B = 2$
- **Code**: Fraction of tests passed, $B = 11$ (for 10 tests)
- **Instruction Following**: Multi-criterion rubric, B up to 32
- **Safety**: Ternary (safe/borderline/unsafe), $B = 3$

6.2 Information by Domain

Each domain contributes:

$$I_{\text{total}} = \sum_{d \in \text{domains}} \sum_{i \in d} K \cdot H(r_i^{(d)})$$

The per-rollout information density varies:

Domain	B	Bits/rollout (max)	vs. Math
Math (binary)	2	1.00	1.0×
Safety (ternary)	3	1.58	1.6×
Code (10 tests)	11	3.46	3.5×
IF (5-criterion)	32	5.00	5.0×

Practical Takeaway

A batch that is 40% math and 30% code by prompt count is not 40%/30% by information. Code rollouts carry 3.5× more bits per rollout. If information is the bottleneck, high-B domains should be oversampled relative to their intrinsic importance.

6.3 Vector Rewards: Scalar Collapse Destroys Information

Modern systems often compute multiple reward signals:

$$\mathbf{r}_j = (r_j^{\text{correct}}, r_j^{\text{format}}, r_j^{\text{length}}, r_j^{\text{style}})$$

Problematic: Collapse to scalar before computing advantage:

$$r_j = \sum_m w_m r_j^{(m)} \implies \text{Information loss}$$

The weighted sum is many-to-one. Different reward vectors can produce the same scalar:

$$H\left(\sum_m w_m r^{(m)}\right) \leq \min\left(\log_2(B_{\text{combined}}), \sum_m H(r^{(m)})\right)$$

Better: Compute per-component advantages:

$$g_j = \sum_m \nabla_\theta \log \pi_\theta(\tau_j) \cdot A_j^{(m)}$$

This preserves:

$$I \leq \sum_m H(r^{(m)})$$

Example 6.1 (Information Destruction)**.** With $M = 10$ independent binary reward signals:

- **Vector approach**: Up to 10 bits per rollout
- **Scalar collapse**: Sum takes values in $\{0, 1, \ldots, 10\}$, giving at most $\log_2(11) = 3.46$ bits
- **Information destroyed**: $10 - 3.46 = 6.54$ bits, or 65% of the signal

6.4 Gradient Interference

Even with preserved information, gradient directions can conflict. If math rewards push θ toward d_1 and code rewards push toward d_2 with $d_1 \cdot d_2 < 0$, the net gradient partially cancels. Information was *present* but not fully *utilized*.

This is an optimization limit, not an information-theoretic one. Manifestations include training instability with heterogeneous batches and potential benefits from per-domain gradient normalization or domain-specific adapters.

7. The Unified Formula and Comparison to Pre-Training

7.1 The Master Equation

Combining all factors:

Key Result

$$\boxed{I_{\text{step}} \leq X \cdot K \cdot M \cdot \log_2(B) \text{ bits}}$$

where:

- X = prompts per batch
- K = rollouts per prompt
- M = independent reward dimensions
- B = reward cardinality per dimension

This ceiling assumes independence. Practical factors that reduce it:

1. **Non-optimal difficulty**: Prompts with $p \neq 0.5$ contribute $< \log_2(B)$ bits
2. **Degenerate groups**: Same-reward groups contribute 0 bits
3. **Reward correlation**: Correlated dimensions don't add full bits

4. **Gradient interference**: Conflicting objectives partially cancel
5. **Optimizer inefficiency**: SGD/Adam don't optimally extract information
6. **Representational limits**: Model may lack capacity to use the signal

7.2 The Pre-Training Comparison

Ord [1] provides a calibration for pre-training information density:

> "GPT-4 has about 10^{12} parameters (so about 3×10^{13} bits [at 32-bit precision]) trained on about 10^{13} tokens of training data, which comes to about 3 bits of model capacity per token of training data. I'm actually surprised by how close this is, as 3 bits of weights per token is the capacity that would be necessary if it was optimally learning all bits of information in the training signal."

Training Paradigm	Bits per Token	Tokens per Signal
Pre-training (next-token prediction)	$\sim$3	1
Distillation (teacher logits)	$\sim$6–10	1
SFT (instruction tuning)	$\sim$2–3	1
RL with binary terminal reward	$\sim$1 per episode	T (thousands)

For $T = 4{,}000$ tokens per episode (DeepSeek-R1 average), RL is approximately 12,000$\times$ less information-dense than pre-training per token generated.

Even with GRPO ($K = 16$) and 5-level rewards ($B = 5$):

$$\text{RL bits/token} = \frac{K \cdot \log_2(B)}{T} = \frac{16 \cdot 2.32}{4{,}000} \approx 0.009$$

Still $\sim$300$\times$ less dense than pre-training.

7.3 Reconciling RL's Effectiveness

Given this massive inefficiency, why does RL work?

Key Result

RL bits are **targeted bits**. They directly optimize the objective we care about (task success, human preference) rather than compressing arbitrary text.
Pre-training learns "what text looks like." RL learns "what good solutions look like." The latter is a much more constrained target.

Ord [1] offers this resolution: "My best guess is that it comes down to the breadth of what the systems are learning. RL is great for learning a narrow task in great depth and bursting through the human-range of abilities without even a kink in the learning curve."

Additional factors:

- The model already has most relevant "knowledge" from pre-training
- RL is fine-tuning, not learning from scratch
- Low-rank adapters suffice because the information budget is inherently limited

8. Practical Implications

8.1 Reward Engineering

Practical Takeaway

Increasing reward cardinality from B to αB gives:

$$\Delta I = K \cdot \log_2(\alpha) \text{ bits/prompt}$$

at zero additional compute cost. Increasing rollouts from K to αK gives similar information gain but costs $\alpha\times$ compute.
Implication: Invest in richer rewards before scaling rollouts.

8.2 Curriculum Learning

Practical Takeaway

Information per prompt is maximized at $p \approx 0.5$:

- Too easy ($p \to 1$): Low entropy, degenerate groups
- Too hard ($p \to 0$): Low entropy, degenerate groups
- Optimal ($p \approx 0.5$): Maximum entropy, minimum degeneracy

Train on problems at the frontier of the model's current ability.

8.3 Batch Composition

Practical Takeaway

Allocate compute proportionally to information rate:

$$\text{Optimal fraction for domain } d \propto \frac{\log_2(B_d) \cdot H(p_d)}{T_d}$$

Domains with rich rewards and short episodes should be oversampled.

8.4 Dynamic Sampling

Practical Takeaway

Skip zero-information groups (DAPO [5]/GRESO [6] approach):

- Filter prompts where all K rollouts received identical rewards
- Refill batches with informative prompts
- Achieves 2×–2.4× speedup with no accuracy loss

8.5 Rollout Count

Practical Takeaway

The "It Takes Two" [7] result suggests 2-GRPO retains 98% of 16-GRPO performance at 12.5% of rollout cost. Consider whether high K is necessary for your application.

9. Summary

The "1 bit per step" claim is **correct** under specific conditions (single rollout, binary terminal reward, scalar advantage) and **incomplete** as a general characterization.

The actual information bound is:

$$I_{\text{step}} \leq X \cdot K \cdot M \cdot \log_2(B)$$

Key levers to increase throughput:

1. **Richer rewards** ($\uparrow B$): Zero compute cost, invest here first
2. **More rollouts** ($\uparrow K$): Linear cost, diminishing returns past $K \approx 2$–8
3. **More prompts** ($\uparrow X$): Memory-limited, high leverage if feasible
4. **Vector rewards** ($\uparrow M$): Preserve structure, avoid scalar collapse
5. **Curriculum**: Maintain $p \approx 0.5$ to maximize entropy
6. **Dynamic sampling**: Skip degenerate groups

The 10^3–$10^6\times$ information density gap with pre-training explains why:

- RL requires far fewer tokens than pre-training
- Low-rank adapters suffice for RL fine-tuning
- RL builds on pre-trained representations rather than learning from scratch

The bits RL provides are targeted at the objective we care about, explaining dramatic task improvements despite their scarcity.

Limitations and Open Questions

1. **Mutual information bounds are not achievability results.** The bound $I(g; \pi^*) \leq 1$ bit tells us the ceiling; it does not guarantee we can extract that bit efficiently.
2. **The optimal policy π^* is not a fixed target.** In practice, we estimate the reward function (via a learned reward model), introducing additional uncertainty not captured in this analysis.
3. **Multi-step learning effects.** Information accumulates across gradient steps. The interaction between steps—how early learning affects what later steps can learn—is not captured by per-step bounds.
4. **Representation learning.** RL may modify internal representations in ways that unlock future learning, providing value beyond the immediate gradient step.

References

[1] T. Ord, "The Extreme Inefficiency of RL for Frontier Models," 2025. https://www.tobyord.com/writing/inefficiency-of-reinforcement-learning

[2] Y. Li, "Information Bandwidth in Reinforcement Learning," 2025. https://richardli.xyz/post/information-bandwidth-rl/

[3] DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," arXiv:2501.12948, 2025.

[4] Z. Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," arXiv:2402.03300, 2024.

[5] Q. Yu et al., "DAPO: An Open-Source LLM Reinforcement Learning System at Scale," arXiv:2503.14476, 2025.

[6] Infini-AI-Lab, "GRESO: GRPO with Efficient Selective Rollout," 2025. https://github.com/Infini-AI-Lab/GRESO

[7] Y. Wu et al., "It Takes Two: Your GRPO Is Secretly DPO," arXiv:2510.00977, 2025.